

FPGA Experiment Report

Experiment 6 — VGA Protocol (Screen Output)

Task: FPGA 6 — VGA interface for screen output

Student 1: Mahmood Stitia

ID: 214032682

Student 2: Saleh Khalil

ID: 213310485

Part 1 — Status update

The table below lists the submitted files and their status. The status is one of *Done* (module and simulation written and passing), *Partially done* (started but not fully working — explained in the notes), or *Not started*.

File / Step	Status	Notes / comments
VGA_Interface.v	Done	
VGA_Interface_tb.v	Done	
Drawer.v	Done	
VGA_Top.v	Done	
task2.xdc	Done	

Issues

Any errors present at the time of submission are described here (per module / simulation, with what the error means).

- VGA_Interface.v: None.
- VGA_Interface_tb.v: None.
- Drawer.v: None.
- VGA_Top.v: None.
- task2.xdc: None.

Part 2 — Simulations and answers

Simulation — VGA_Interface

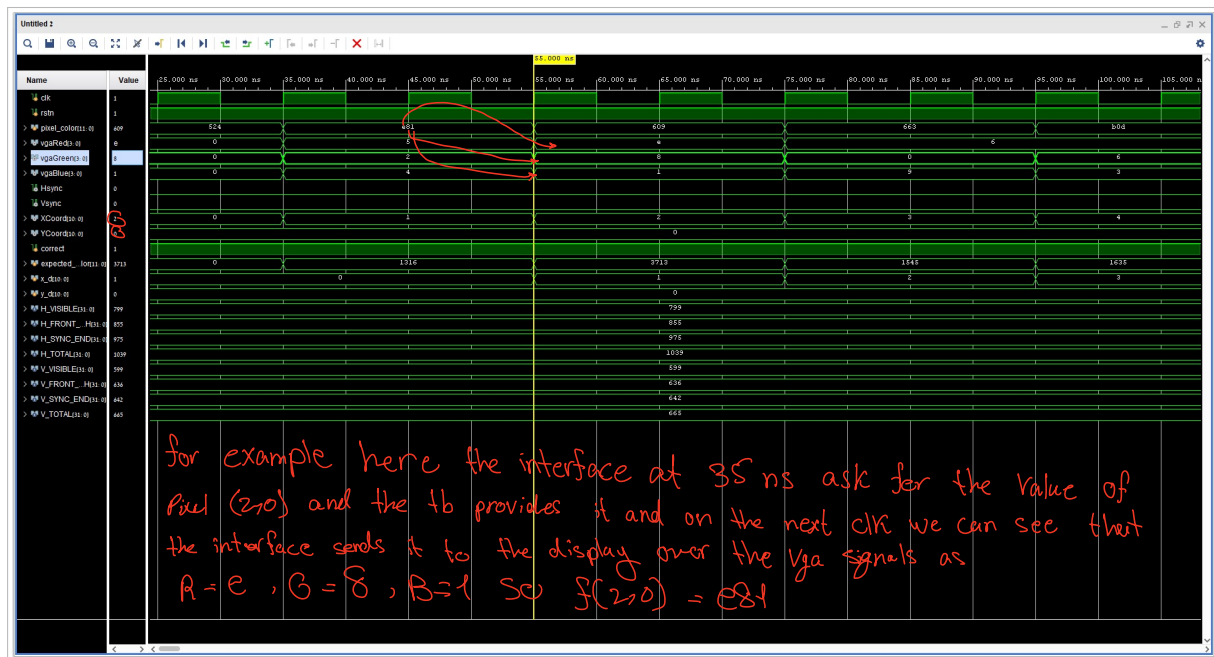
The design targets the **800×600 @ 72 Hz** VGA mode. To reproduce this in simulation, the test-bench drives a 100 MHz clock and, for every pixel inside the visible window (columns 0 to 799 and rows 0 to 599), applies a pseudo-random color to the `pixel_color` input, while forcing it to zero (black) everywhere outside that window. Two counters track the beam position: `h_count` runs 0 to 1039 (1040 pixel-clocks per line) and `v_count` runs 0 to 665 (666 lines per frame). Only the 800×600 window is visible; outside it the outputs are blanked to black.

Pixel-clock and Blanking Considerations

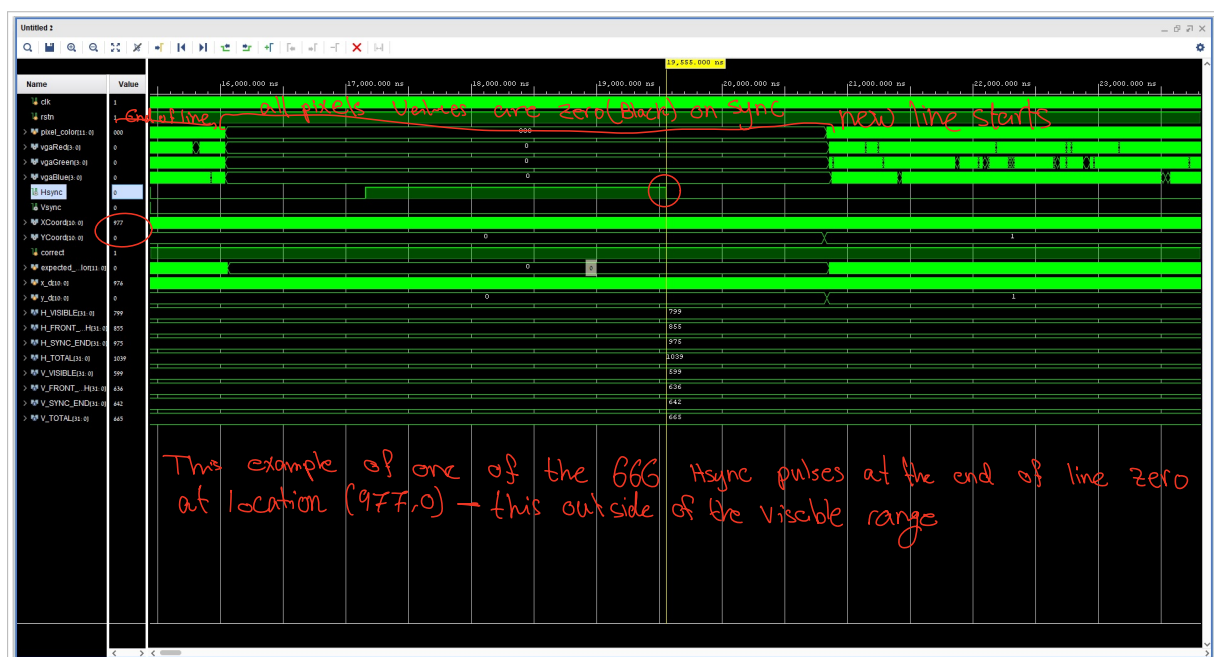
The 50 MHz pixel rate is produced with a *clock enable* rather than a separate clock: a register (`pix_en`) toggles on every system-clock edge, so it is high once every two cycles, and the counters and outputs advance only on those ticks. This keeps the whole design in the single 100 MHz clock domain while updating at the 50 MHz pixel rate. The sync and color outputs are registered (flip-flops) so the VGA pins remain stable, and outside the 800×600 visible window the color outputs are forced to $\{R,G,B\} = 0$ (black)

Signals (top to bottom in all 4 waveforms):

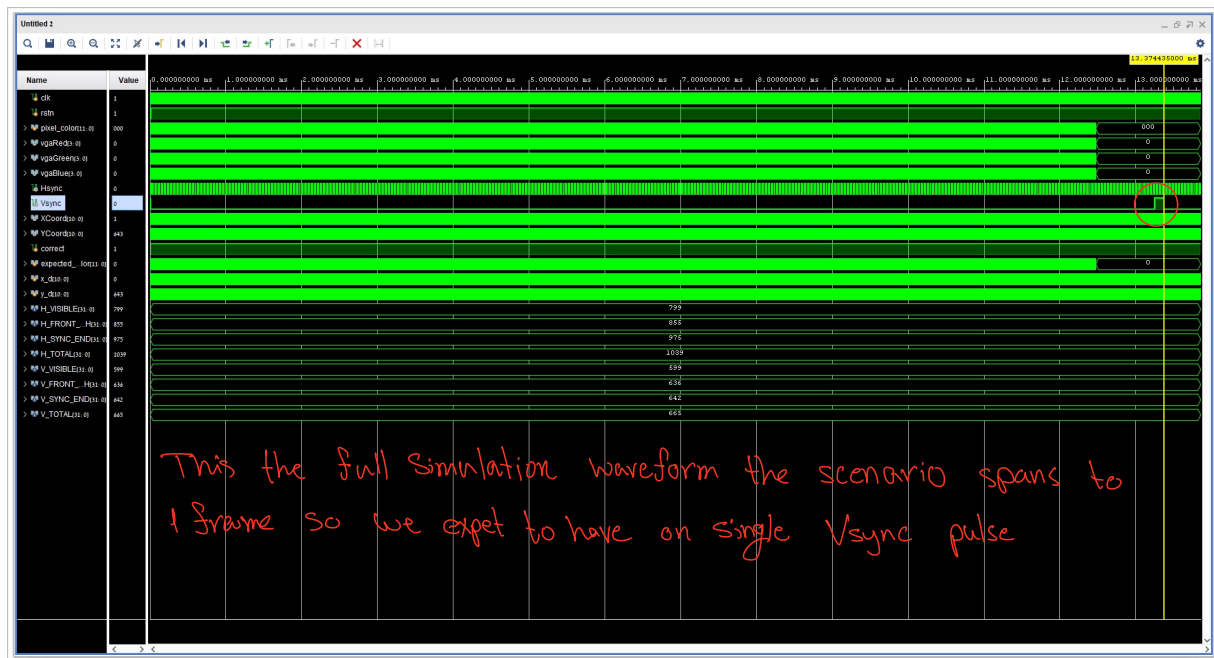
- `clk` - 100 MHz system clock (test-bench driven).
- `rstn` - active-low reset.
- `pixel_color[11:0]` - test-bench color input: pseudo-random inside the visible window, 0 in blanking.
- `vgaRed/vgaGreen/vgaBlue[3:0]` - color outputs (forced to 0 in blanking).
- `Hsync / Vsync` - sync outputs (positive polarity).
- `XCoord[10:0] / YCoord[10:0]` - current pixel position (`h_count` / `v_count`).
- `expected_pixel_color[11:0]` - test-bench reference color for the current pixel.
- `correct` - self-check flag; stays 1 while the outputs match the reference.
- `x_d[10:0] / y_d[10:0]` - `XCoord/YCoord` delayed by one pixel to align with the registered outputs.
- `H_VISIBLE`, `H_FRONT_PORCH`, `H_SYNC_END`, `H_TOTAL` - horizontal timing constants (799, 855, 975, 1039).
- `V_VISIBLE`, `V_FRONT_PORCH`, `V_SYNC_END`, `V_TOTAL` - vertical timing constants (599, 636, 642, 665).



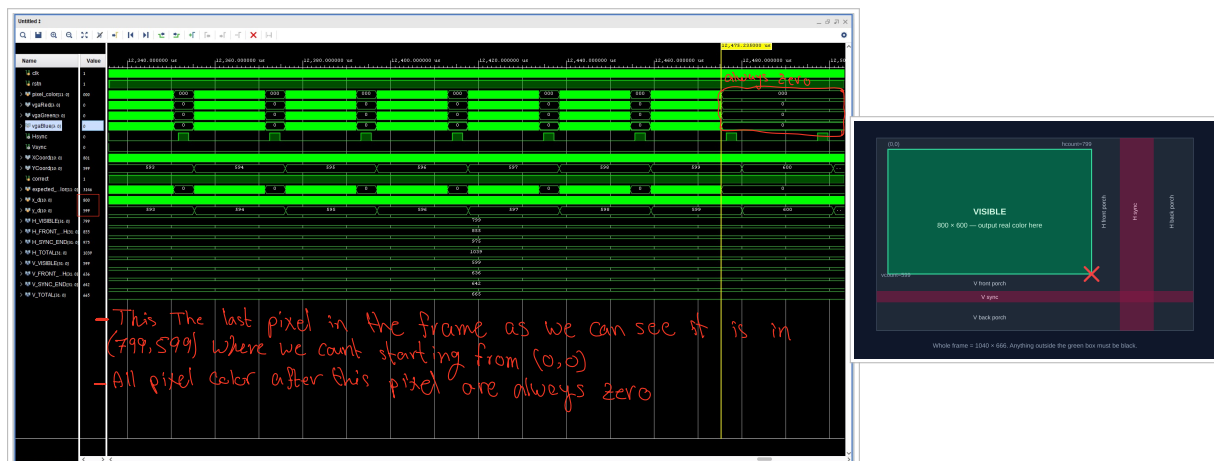
At 35 ns the interface presents the coordinate $(XCoord, YCoord) = (2, 0)$; the test-bench supplies the random color for that pixel, and on the next pixel clock the interface drives it onto the VGA outputs — $vgaRed = E$, $vgaGreen = 8$, $vgaBlue = 1$, i.e. $f(2,0) = E81$. The one-clock gap between the coordinate and the color is the registered-output pipeline delay.



One of the 666 horizontal-sync pulses, shown here at the end of line 0. The marked sample is at $(XCoord, YCoord) = (977, 0)$, which lies in the horizontal back porch — beyond the visible range 0 to 799 — so the color outputs are blanked to black (000). When the line ends the horizontal counter wraps and the next line begins.



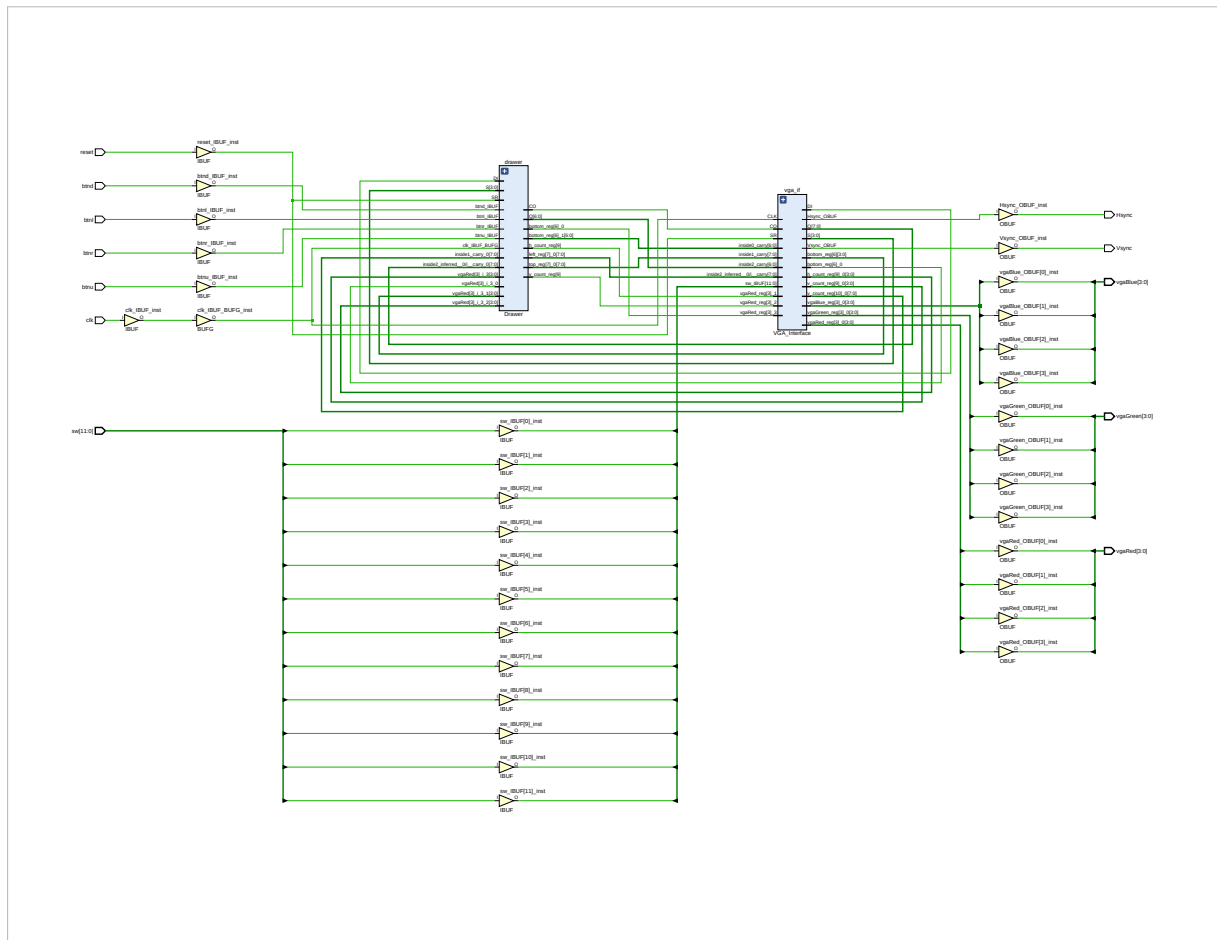
Full-frame view of the simulation. The run spans exactly one frame (1040×666 pixel-clocks), so a single vertical-sync (V_{sync}) pulse appears near the end, confirming that one complete frame was generated.



Same boundary, two views. Left: the simulation at the last visible pixel, $(xCoord, yCoord) = (799, 599)$ — the final cell of the 800×600 window; right after it every color output drops to 000. Right: the frame's region map, where the red cross marks the bottom-right corner of the green visible box at exactly $hcount = 799$, $vcount = 599$. Both views describe the same point: once the beam passes $(799, 599)$ it leaves the green region and enters the blanking area (front/back porches and sync), which the VGA standard forces to black. The waveform is simply the time-domain trace of this spatial map.

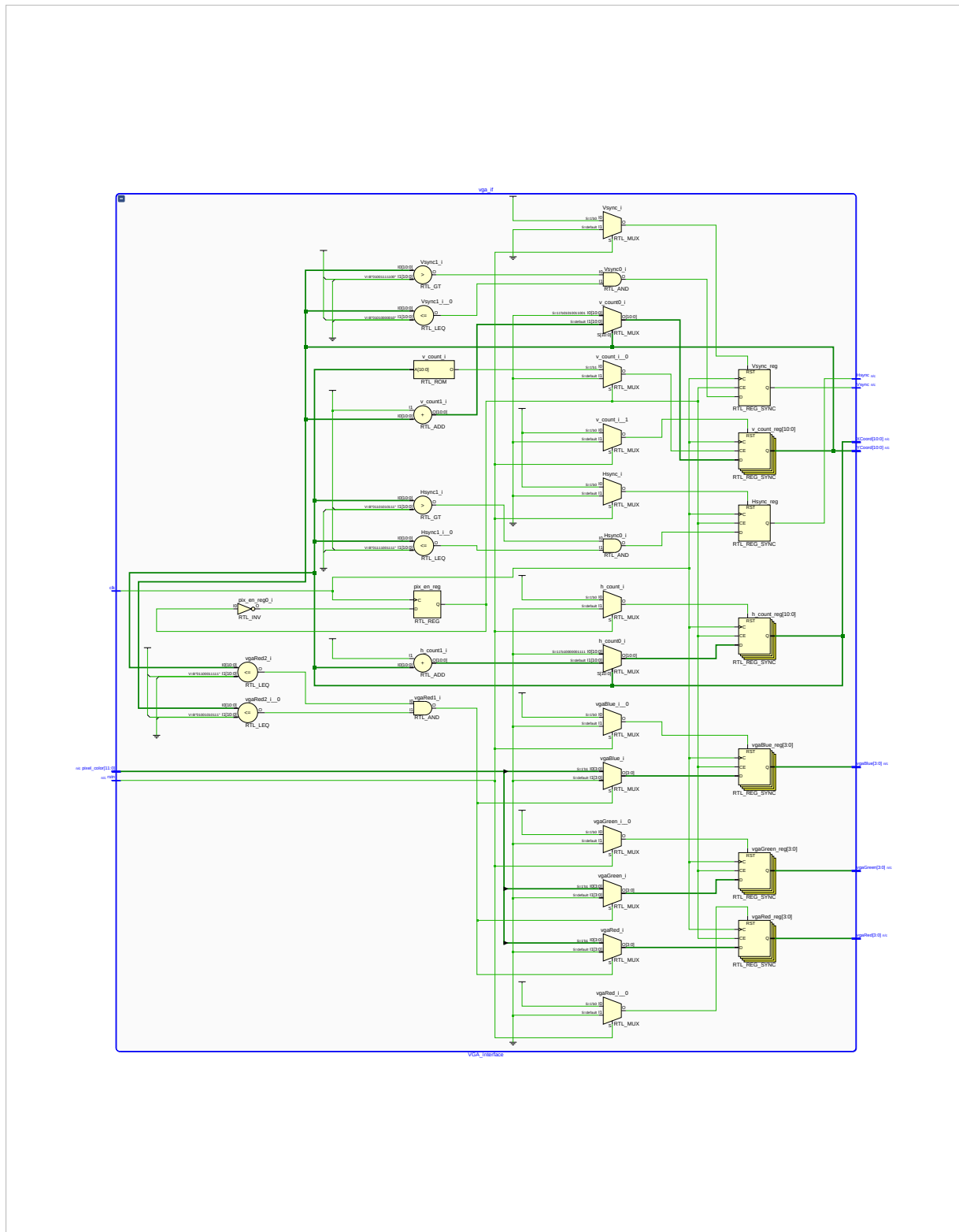
Elaborated Design

Synthesis schematic



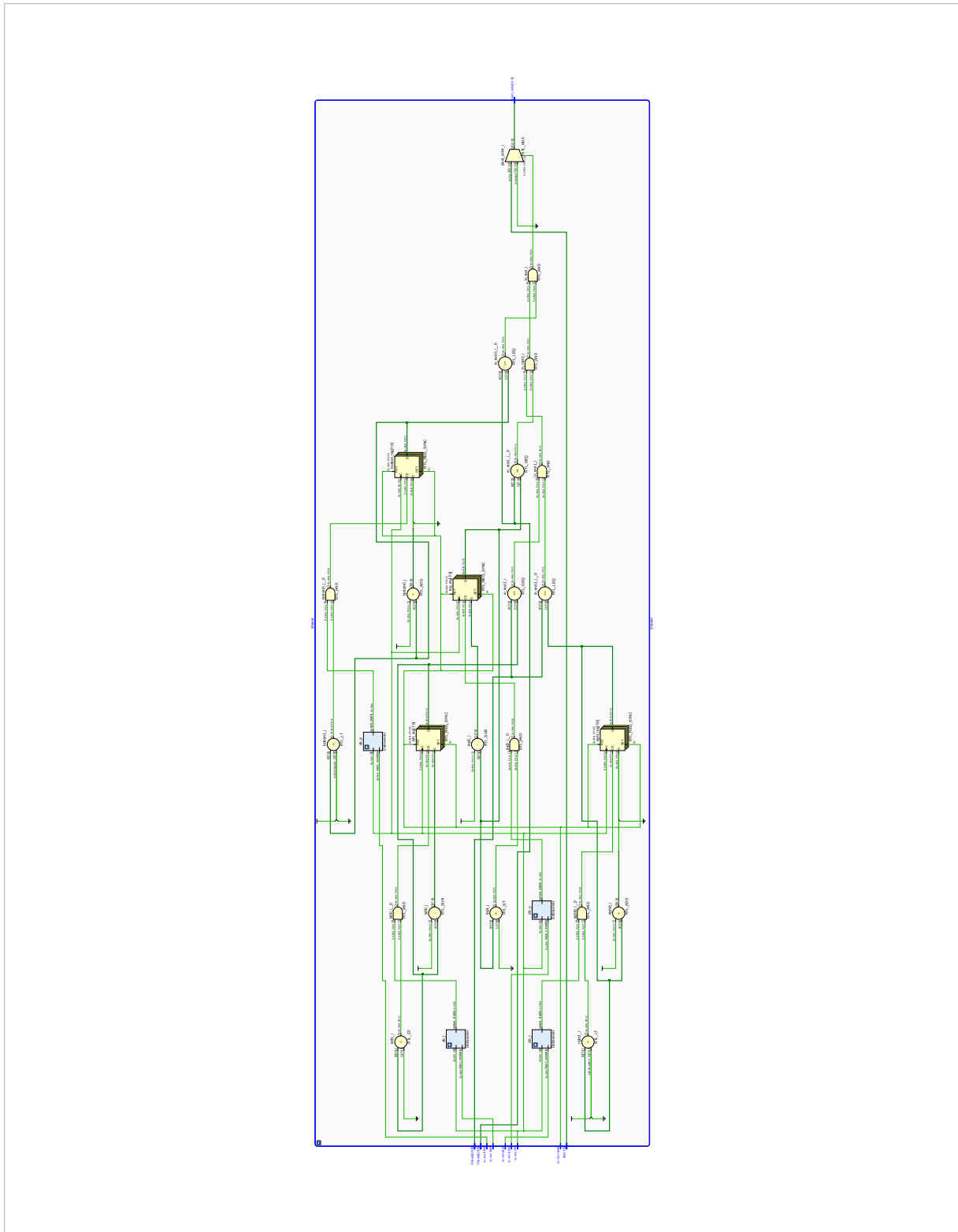
Post-synthesis schematic.

VGA_Interface



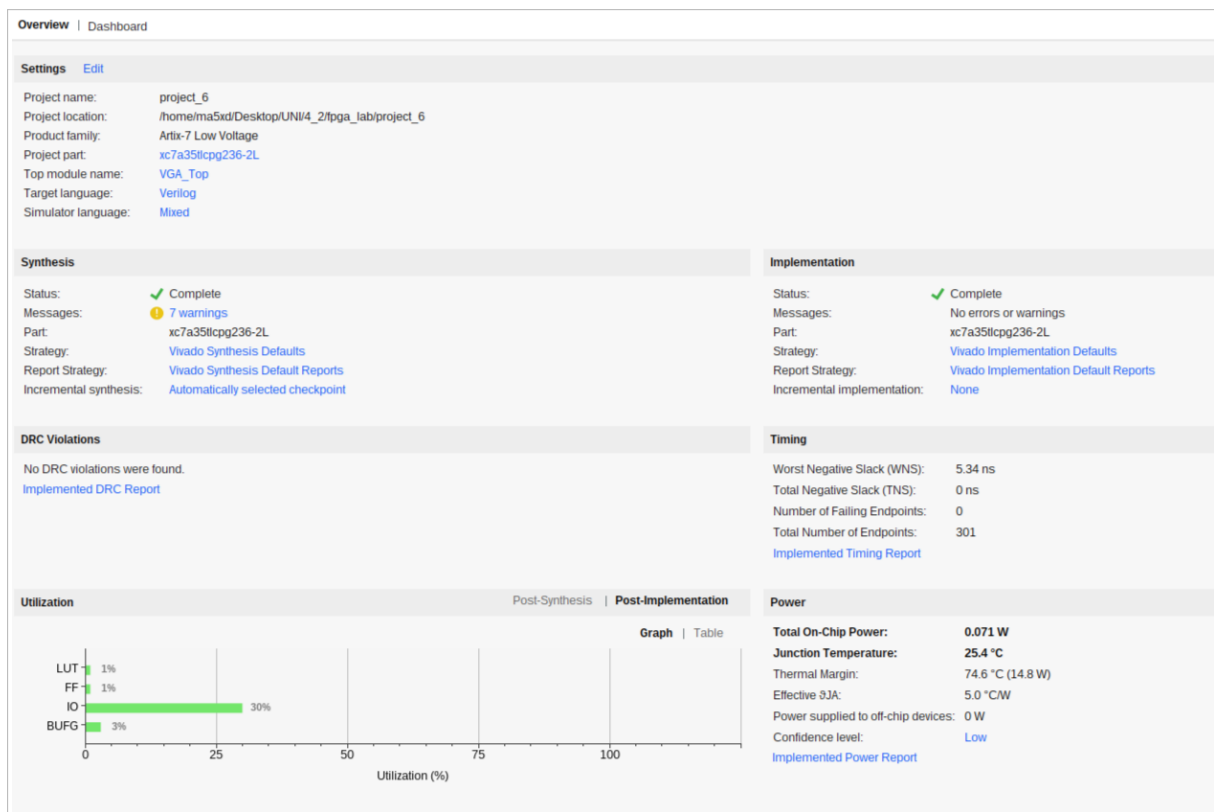
RTL-elaborated schematic of the VGA_Interface.

Drawer



RTL-elaborated schematic of the Drawer.

Project summary



Post-implementation project summary.

Timing summary

Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS): 5.340 ns		Worst Hold Slack (WHS): 0.236 ns		Worst Pulse Width Slack (WPWS): 4.500 ns	
Total Negative Slack (TNS): 0.000 ns		Total Hold Slack (THS): 0.000 ns		Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	
Total Number of Endpoints: 301		Total Number of Endpoints: 301		Total Number of Endpoints: 156	
All user specified timing constraints are met.					

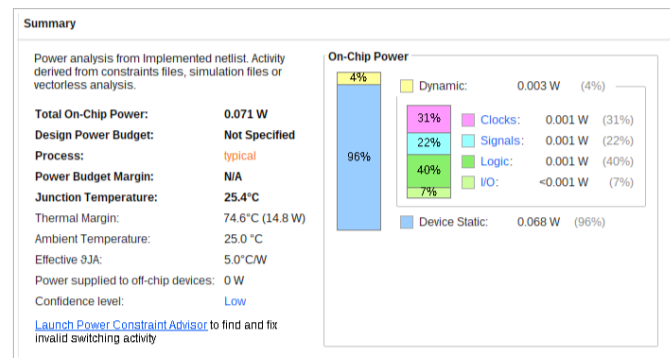
Design Timing Summary after implementation.

Critical Path

Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement
↳ Path 1	5.340	4	12	vga_if/h_count_reg[3]/C	vga_if/vgaBlue_reg[0]/D	4.613	1.753	2.860	10.0
↳ Path 2	5.354	4	12	vga_if/h_count_reg[3]/C	vga_if/vgaBlue_reg[1]/D	4.596	1.753	2.843	10.0
↳ Path 3	5.361	4	12	vga_if/h_count_reg[3]/C	vga_if/vgaBlue_reg[2]/D	4.592	1.753	2.839	10.0
↳ Path 4	5.368	4	12	vga_if/h_count_reg[3]/C	vga_if/vgaRed_reg[3]/D	4.585	1.753	2.832	10.0
↳ Path 5	5.478	4	12	vga_if/h_count_reg[3]/C	vga_if/vgaBlue_reg[3]/D	4.473	1.753	2.720	10.0

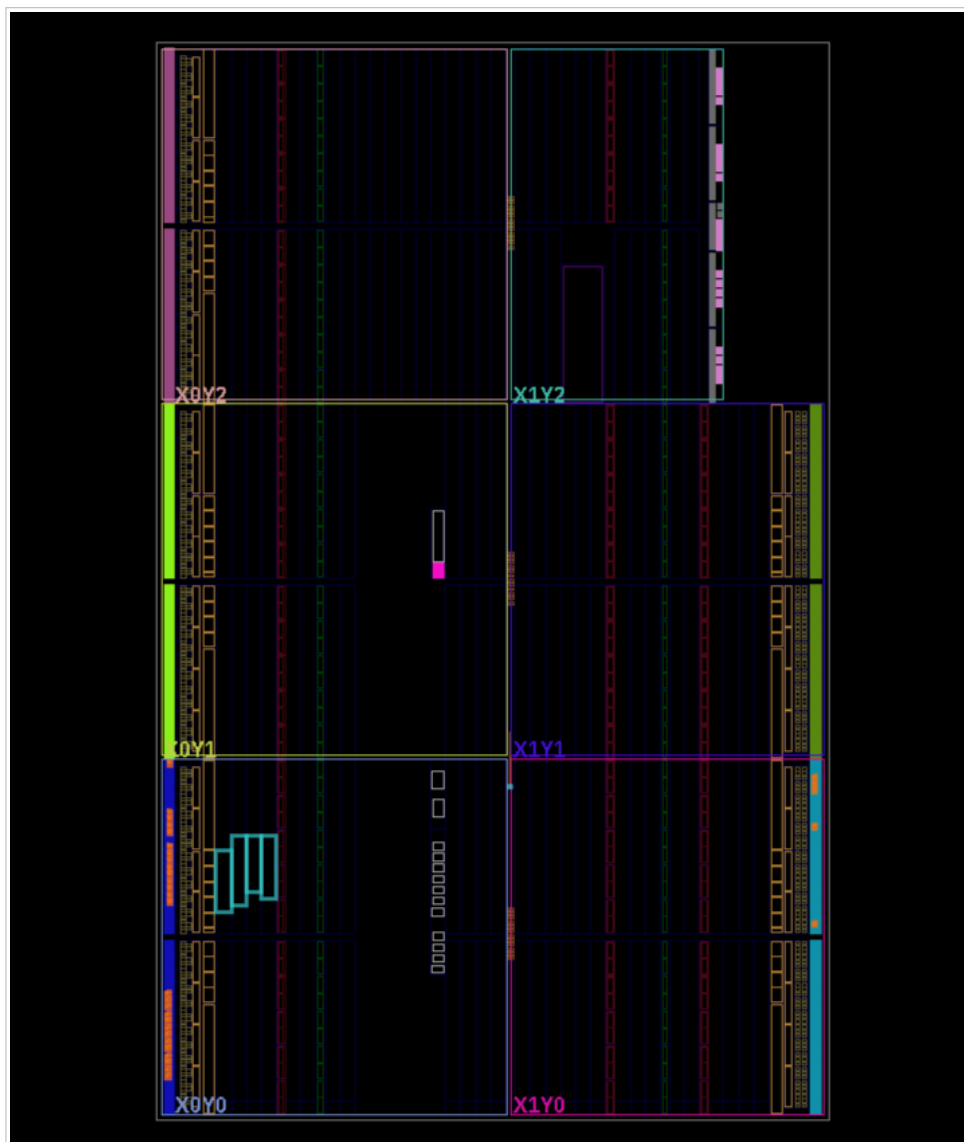
Intra-Clock Paths report (worst path, slack, logic levels and delay).

Power summary



Post-implementation power estimate.

Device



Device (floorplan) view of the placed-and-routed design on the Artix-7 (XC7A35T).

Working Device

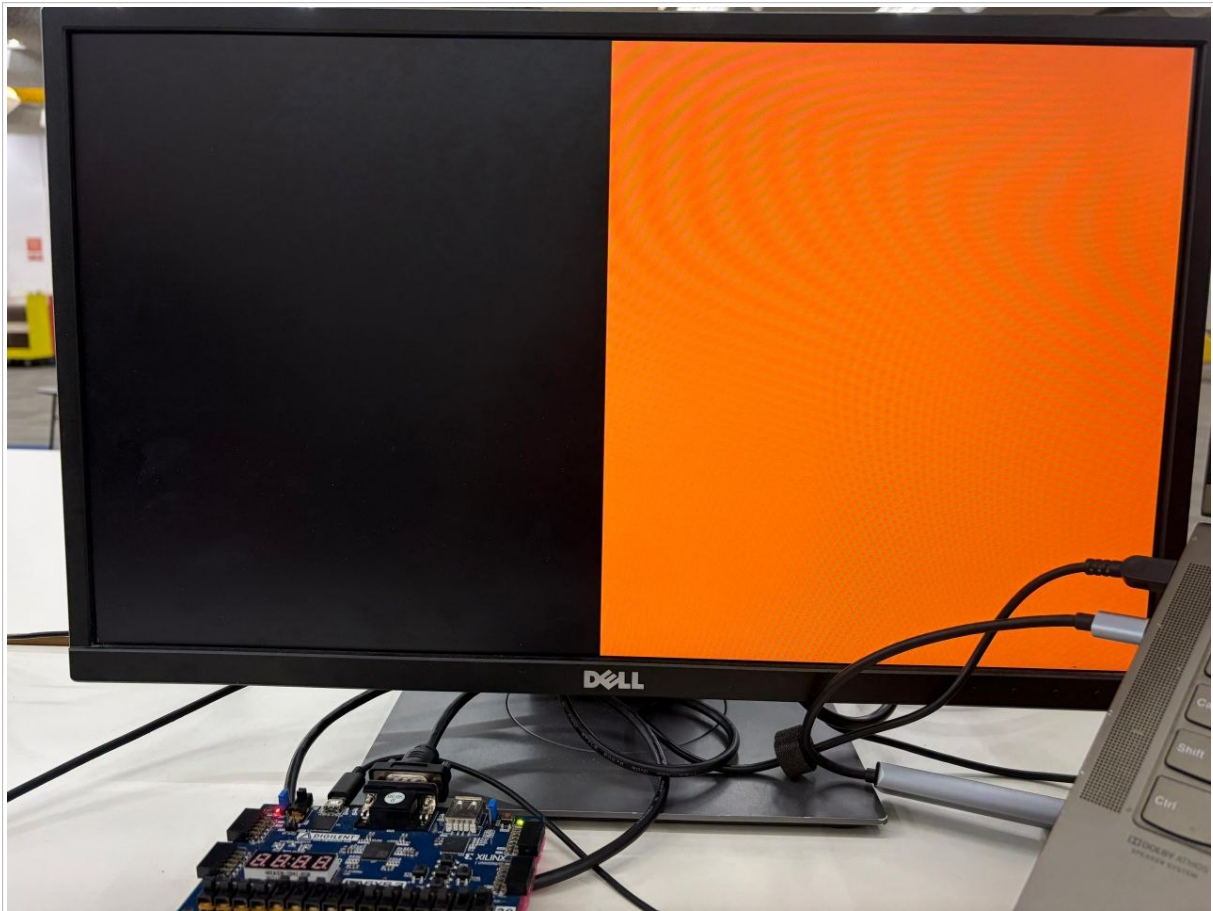


image of the working display

Appendix — Verilog Source Code

VGA_Interface.v

```

1  'define INC(x, max) ((x == max-1)? 0 : (x + 1))
2  module VGA_Interface(
3      input  wire      clk,
4      input  wire      rstn,
5      input  wire [11:0] pixel_color,
6      output reg  [3:0] vgaRed,
7      output reg  [3:0] vgaGreen,
8      output reg  [3:0] vgaBlue,
9      output reg      Hsync,
10     output reg      Vsync,
11     output wire [10:0] XCoord,
12     output wire [10:0] YCoord
13 );
14
15     reg [10:0] h_count;
16     reg [10:0] v_count;
17
18     localparam H_VISIBLE = 799, H_FRONT_PORCH = 855, H_SYNC_END = 975,
19               H_TOTAL = 1039;
20     localparam V_VISIBLE = 599, V_FRONT_PORCH = 636, V_SYNC_END = 642,
21               V_TOTAL = 665;
22
23     // 50 MHz pixel-clock (divide 100 MHz by 2)
24     reg pclk = 0;
25     always @(posedge clk) pclk <= ~pclk;
26
27     // Display logic (registered outputs)
28     always @(posedge pclk) begin
29         if (!rstn) begin
30             Hsync <= 1'b0; Vsync <= 1'b0;
31             vgaRed <= 4'h0; vgaGreen <= 4'h0; vgaBlue <= 4'h0;
32         end else begin
33             Hsync <= (h_count > H_FRONT_PORCH) && (h_count <= H_SYNC_END);
34             Vsync <= (v_count > V_FRONT_PORCH) && (v_count <= V_SYNC_END);
35             if ((h_count <= H_VISIBLE) && (v_count <= V_VISIBLE)) begin
36                 vgaRed <= pixel_color[11:8];
37                 vgaGreen <= pixel_color[7:4];
38                 vgaBlue <= pixel_color[3:0];
39             end else begin
40                 vgaRed <= 4'h0; vgaGreen <= 4'h0; vgaBlue <= 4'h0;
41             end
42         end
43     end
44
45     // Coordinate counters
46     always @(posedge pclk) begin
47         if (!rstn) begin
48             h_count <= 0; v_count <= 0;
49         end else begin
50             h_count <= 'INC(h_count, H_TOTAL+1);
51             if (h_count == H_TOTAL)
52                 v_count <= 'INC(v_count, V_TOTAL+1);
53         end
54     end
55
56     assign XCoord = h_count;
57     assign YCoord = v_count;
58 endmodule

```

VGA_Interface_tb.v

```

1  module VGA_Interface_tb;
2
3      reg clk;
4      reg rstn;
5      reg [11:0] pixel_color;
6      wire [3:0] vgaRed;
7      wire [3:0] vgaGreen;
8      wire [3:0] vgaBlue;
9      wire Hsync;
10     wire Vsync;
11     wire [10:0] XCoord;
12     wire [10:0] YCoord;
13
14     reg correct;
15
16     always #5 clk = ~clk; // 100 MHz clk
17
18     VGA_Interface vga_if(
19         // Inputs
20         .clk(clk),
21         .rstn(rstn),
22         .pixel_color(pixel_color),
23         // Outputs - to VGA pins
24         .vgaRed(vgaRed),
25         .vgaGreen(vgaGreen),
26         .vgaBlue(vgaBlue),
27         .Hsync(Hsync),
28         .Vsync(Vsync),
29         // Outputs - to Drawer
30         .XCoord(XCoord),
31         .YCoord(YCoord)
32     );
33
34     localparam H_VISIBLE = 799, H_FRONT_PORCH = 855, H_SYNC_END = 975,
35                H_TOTAL = 1039;
36     localparam V_VISIBLE = 599, V_FRONT_PORCH = 636, V_SYNC_END = 642,
37                V_TOTAL = 665;
38
39     reg [11:0] expected_pixel_color;
40     reg [10:0] x_d, y_d;
41
42     always @(posedge clk) begin
43         if(vga_if.pix_en == 1) begin
44             expected_pixel_color <= pixel_color;
45             x_d <= XCoord;
46             y_d <= YCoord;
47             pixel_color <= (XCoord <= H_VISIBLE && YCoord <= V_VISIBLE)?
48                 $random : 12'h000;
49             correct <= ({vgaRed, vgaGreen, vgaBlue} === ((x_d <= H_VISIBLE
50                 && y_d <= V_VISIBLE) ? expected_pixel_color : 12'h000)) &&
51                 (Hsync === ((x_d > H_FRONT_PORCH) && (x_d <=
52                 H_SYNC_END))) &&
53                 (Vsync === ((y_d > V_FRONT_PORCH) && (y_d <=
54                 V_SYNC_END))) &&
55                 correct;
56         end
57     end
58 end

```

```
56     initial begin
57         if ($test$plusargs("vcd")) begin
58             $dumpfile("VGA_Interface_tb.vcd");
59             $dumpvars(0, VGA_Interface_tb);
60         end
61
62         clk = 0;
63         correct = 1;
64         rstn = 0;
65         pixel_color = 12'h000;
66         expected_pixel_color = 12'h000;
67         x_d = 0;
68         y_d = 0;
69
70         // reset for 20 ns
71         #20 rstn = 1;
72         correct = 1;
73
74         repeat ((H_TOTAL+1) * (V_TOTAL+1) * 2) #10; // run for 1 frames
75
76         if (correct) begin
77             $display("Test passed!!");
78         end else begin
79             $display("Test failed!");
80         end
81         $finish;
82     end
83
84 endmodule
```

Drawer.v

```

1  `timescale 1ns/10ps
2  // Module Name: Drawer - pixel color / movable rectangle logic
3
4  module Drawer(
5      input wire      clk,
6      input wire      reset,
7      input wire [11:0] sw,
8      input wire      btneu,
9      input wire      btnd,
10     input wire      btntl,
11     input wire      btnr,
12     input wire [10:0] XCoord,
13     input wire [10:0] YCoord,
14     output wire [11:0] pixel_color
15 );
16
17     // 800x600 visible region downsampled by 8
18     localparam integer GRID_W = 100;
19     localparam integer GRID_H = 75;
20     localparam integer CX      = GRID_W / 2; // 50
21     localparam integer CY      = GRID_H / 2; // 37
22
23     // 20-bit debouncer
24     wire btneu_pulse, btnd_pulse, btntl_pulse, btnr_pulse;
25     Debouncer #(.COUNTER_BITS(20)) db_u (.clk(clk), .input_unstable(btneu), .
        output_stable(btneu_pulse));
26     Debouncer #(.COUNTER_BITS(20)) db_d (.clk(clk), .input_unstable(btnd), .
        output_stable(btnd_pulse));
27     Debouncer #(.COUNTER_BITS(20)) db_l (.clk(clk), .input_unstable(btntl), .
        output_stable(btntl_pulse));
28     Debouncer #(.COUNTER_BITS(20)) db_r (.clk(clk), .input_unstable(btnr), .
        output_stable(btnr_pulse));
29
30     // Rectangle edges in coarse-grid units (inclusive)
31     reg [7:0] top, bottom, left, right;
32
33     always @(posedge clk) begin
34         if (reset) begin
35             top <= CY; bottom <= CY; left <= CX; right <= CX;
36         end else begin
37             if (btneu_pulse && top > 0) top <= top - 1'b1;
38             if (btnd_pulse && bottom < GRID_H - 1) bottom <= bottom + 1'b1;
39             if (btntl_pulse && left > 0) left <= left - 1'b1;
40             if (btnr_pulse && right < GRID_W - 1) right <= right + 1'b1;
41         end
42     end
43
44     // Coarse-grid coordinate (divide by 8)
45     wire [7:0] x_ds = XCoord[10:3];
46     wire [7:0] y_ds = YCoord[10:3];
47
48     wire inside = (x_ds >= left) && (x_ds <= right)
49                 && (y_ds >= top) && (y_ds <= bottom);
50
51     assign pixel_color = inside ? sw : 12'h000;
52
53 endmodule

```

VGA_Top.v

```
1  `timescale 1ns/10ps
2  // Module Name: VGA_Top - connects VGA_Interface and Drawer
3
4  module VGA_Top(
5      input wire          clk,
6      input wire          reset,
7      input wire [11:0] sw,
8      input wire          btneu,
9      input wire          btnd,
10     input wire          btnl,
11     input wire          btnr,
12     output wire [3:0] vgaRed,
13     output wire [3:0] vgaGreen,
14     output wire [3:0] vgaBlue,
15     output wire          Hsync,
16     output wire          Vsync
17 );
18
19     wire [10:0] XCoord;
20     wire [10:0] YCoord;
21     wire [11:0] pixel_color;
22
23     VGA_Interface vga_if(
24         .clk(clk), .rstn(~reset), .pixel_color(pixel_color),
25         .vgaRed(vgaRed), .vgaGreen(vgaGreen), .vgaBlue(vgaBlue),
26         .Hsync(Hsync), .Vsync(Vsync),
27         .XCoord(XCoord), .YCoord(YCoord)
28     );
29
30     Drawer drawer(
31         .clk(clk), .reset(reset), .sw(sw),
32         .btneu(btneu), .btnd(btnd), .btnl(btnl), .btnr(btnr),
33         .XCoord(XCoord), .YCoord(YCoord),
34         .pixel_color(pixel_color)
35     );
36
37 endmodule
```